

Handwritten Digit Recognition Using CNN

(Software Requirements Specification)

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Handwritten Digit Recognition System developed using Convolutional Neural Networks (CNN). The system is designed to recognize handwritten digits (0–9) from user-uploaded images and provide real-time predictions.

1.2 Document Conventions

- Headings are formatted in bold.
- Requirements are labeled using unique identifiers (e.g., REQ-1, REQ-2).
- All technical terms follow standard machine learning and software engineering conventions.

1.3 Intended Audience and Reading Suggestions

- This document is intended for:
 - a. Developers: For implementation details
 - b. Project evaluators: For understanding system functionality
 - c. Testers: For validating requirements
 - d. Students/Users: For system usage understanding
- Reading order:
 - a. Introduction
 - b. Overall Description
 - c. System Features
 - d. Non-functional Requirements

1.4 Product Scope

The system is a web-based application that uses a trained CNN model to recognize handwritten digits. It allows users to upload an image and receive an instant prediction.

- Objectives:
 - Automate digit recognition.
 - Achieve high accuracy using deep learning.
 - Provide an interactive user interface.

2. Overall Description

2.1 Product Perspective

This system is a standalone application combining:

- A trained CNN model (TensorFlow)
- A backend server (Flask)
- A frontend interface (HTML/CSS)

It is not dependent on any existing system but can be extended for larger applications such as document processing systems.

2.2 Product Functions

The system performs the following functions:

- Accept image input from user
- Preprocess the image
- Predict digit using CNN model
- Display prediction result
- Handle invalid inputs

2.3 User Classes and Characteristics

General User

- Uploads image and receives prediction
- Basic computer knowledge

Developer

- Maintains system
- Knowledge of Python, TensorFlow, Flask

2.4 Operating Environment

- Operating System: Windows / Linux / MacOS
- Backend: Python (TensorFlow, Flask)
- Frontend: HTML, CSS
- Browser: Chrome, Edge, Firefox

2.5 Design and Implementation Constraints

- Model input size fixed at 28×28 pixels
- Requires Python environment
- Dependent on TensorFlow library
- Internet required for deployment (optional)

2.6 User Documentation

The following user documentation components will be provided along with the system:

1. User Guide:

A basic user manual describing how to access the web application, upload an image, and view the predicted digit output.

2. Installation Guide:

Step-by-step instructions for setting up the project locally, including Python installation, required libraries, and running the Flask application.

3. Online Help (Interface-Based Guidance):

The web interface provides minimal on-screen instructions such as file upload prompts and prediction display messages for ease of use.

4. Technical Documentation:

Basic documentation for developers, including model loading, preprocessing steps, and application workflow for future enhancements.

All documentation is provided in digital format (PDF and text-based files) and follows standard academic documentation practices.

2.7 Assumptions and Dependencies

- Input images are handwritten digits
- Images are reasonably clear and cropped for centered digit
- Model is pre-trained before deployment
- Depends on TensorFlow and Flask libraries

3. External Interface Requirements

3.1 User Interfaces

1. Simple web interface with:
 - File upload option
 - Predict button
 - Display result section
2. Clean UI with minimal interaction steps

3.2 Hardware Interfaces

- Standard computer system

- No special hardware required

3.3 Software Interfaces

- TensorFlow (for model prediction)
- Flask (backend server)
- PIL (image processing)
- NumPy (data handling)

3.4 Communications Interfaces

- HTTP protocol for web communication
- Localhost or deployed server access

4. System Features

4.1 Image Upload and Prediction

4.1.1 Description and Priority

Allows users to upload an image and receive a predicted digit.
(**Priority:** High)

4.1.2 Stimulus/Response Sequences

- The user uploads an image.
- System processes the image.
- The model predicts the digit.
- Result displayed.

4.1.3 Functional Requirements

REQ-1: System shall allow user to upload an image file

REQ-2: System shall preprocess the uploaded image

REQ-3: System shall pass image to CNN model

REQ-4: System shall display predicted digit

REQ-5: System shall handle invalid image input

4.2 Model Processing

4.2.1 Description

- Handles prediction logic using trained CNN

4.2.2 Requirements

REQ-6: System shall load trained model

REQ-7: System shall normalize input image

REQ-8: System shall output prediction probabilities

5. Nonfunctional Requirements

5.1 Performance Requirements

- Prediction response time < 2 seconds
- System should handle multiple requests efficiently

5.2 Safety Requirements

- No physical safety risks
- Prevent invalid file processing

5.3 Security Requirements

- Validate uploaded files
- Prevent malicious inputs
- No sensitive data storage

5.4 Software Quality Attributes

- Accuracy: High prediction accuracy (~97–98%)

- Usability: Simple interface
- Reliability: Consistent predictions
- Maintainability: Modular code structure

6. Project Plan

6.1 Team Members

- Mayuresh Khedkar (EN23CS301608)
 - Responsibilities: Requirement analysis, system design, CNN model development, backend implementation, frontend integration, testing, and documentation.

6.2 Division of Work

As the project is carried out individually, all tasks are managed by one developer and organized phase-wise for systematic development.

- Model Development: CNN training
- Backend Development: Flask integration
- Frontend Design: HTML/CSS
- Testing: Model validation and UI testing

6.3 Time Schedule

Week	Phase	Description
Week 1	Project Planning & Research	Understanding problem statements, studying CNN basics, exploring MNIST dataset, and finalizing technology stack (TensorFlow, Flask, HTML/CSS).
Week 2	Data Preparation & Preprocessing	Loading MNIST dataset, normalizing pixel values, reshaping images (28×28×1), splitting dataset, and visualizing sample data.
Week 3	CNN Model Design	Designing CNN architecture including convolution layers, pooling layers, activation functions (ReLU), and output layer (Softmax).
Week 4	Model Training	Training the CNN model, monitoring accuracy and loss, tuning parameters, and saving the trained model.
Week 5	Model Optimization	Improving model performance through dataset augmentation, reducing overfitting, and finalizing the best model.
Week 6	Backend Development	Setting up Flask environment, loading trained model, creating API for predictions, and implementing preprocessing logic.

Week 7	Backend Integration	Integrating model with Flask API, handling image uploads, returning predictions, and debugging backend issues.
Week 8	Frontend Development	Designing user interface using HTML/CSS, implementing image upload form, displaying prediction results, and applying UI styling.
Week 9	Testing & Debugging	Testing system with various inputs, fixing errors, handling edge cases, and optimizing performance.
Week 10	Documentation & Finalization	Preparing SRS, PPT, project report, capturing screenshots, and practicing final demonstration.

Appendix A: Glossary

- **CNN (Convolutional Neural Network):** A deep learning model used for image classification by extracting features from images.
- **MNIST Dataset:** A dataset of handwritten digits (0–9) used for training and testing image recognition models.
- **TensorFlow:** An open-source framework used for building and training machine learning models.
- **Flask:** A lightweight Python framework used to create web applications and APIs.
- **ReLU (Rectified Linear Unit):** An activation function that outputs zero for negative inputs and keeps positive values unchanged.
- **Softmax:** An activation function that converts model outputs into probability values for classification.
- **Epoch:** One complete cycle of training where the model processes the entire dataset once.
- **Normalization:** The process of scaling input data values to a standard range (usually 0 to 1).
- **Inference:** The process of using a trained model to make predictions on new data.
- **Overfitting:** A situation where the model performs well on training data but poorly on unseen data.

Appendix B: TBD List

- Dataset expansion methods
- Deployment on cloud